

Informe del Sistema Distribuido de Archivos Basado en Etiquetas

Mauro Eduardo Campver Barrios C411

Gabriel Alonso Coro C412

30 de noviembre de 2025

Índice

1. Arquitectura o el Problema de Cómo Diseñar el Sistema	3
1.1. Organización del Sistema Distribuido	3
1.2. Roles del Sistema	3
1.3. Distribución de Servicios en Docker Swarm	3
2. Procesos o el Problema de Cuántos Programas o Servicios Posee el Sistema	4
2.1. Tipos de Procesos dentro del Sistema	4
2.2. Organización o Agrupación de los Procesos	5
2.3. Tipo de Patrón de Diseño con Respecto al Desempeño	5
3. Comunicación o el Problema de Cómo Enviar Información Mediante la Red	5
3.1. Tipo de Comunicación	5
3.2. Comunicación Cliente-Servidor y Servidor-Servidor	6
3.3. Comunicación entre Procesos y Patrones de Mensajes	6
4. Coordinación o el Problema de Poner Todos los Servicios de Acuerdo	7
4.1. Acceso Exclusivo a Recursos y Condiciones de Carrera	7
4.2. Sincronización de Acciones (Ordenación de Eventos)	7
4.3. Toma de Decisiones Distribuidas (Detección de Conflicto)	8
5. Nombrado y Localización o el Problema de Dónde se Encuentra un Recurso y Cómo Llegar al Mismo	8
5.1. Identificación de los Datos y Servicios	8
5.2. Ubicación de los Datos y Servicios	8
5.3. Localización de los Datos y Servicios	9

6. Consistencia y Replicación o el Problema de Solucionar los Problemas que Surgen a Partir de Tener Varias Copias de un Mismo Dato en el Sistema	9
6.1. Distribución de los Datos y Réplicas	9
6.2. Confiabilidad de las Réplicas de los Datos (Quorum)	10
6.3. Consistencia de los Metadatos y Manejo de Particiones	10
7. Tolerancia a Fallas o el Problema de la Robustez del Sistema	11
7.1. Respuesta a Errores y Detección de Fallos	11
7.2. Nivel de Tolerancia a Fallos Esperado	11
7.3. Fallos Parciales, Incorporación y Particiones	12
8. Seguridad o el Problema de Qué Tan Vulnerable es el Diseño	12
8.1. Seguridad con Respecto al Diseño	12
8.2. Seguridad con Respecto a la Comunicación	13
8.3. Autorización y Autenticación	13

1. Arquitectura o el Problema de Cómo Diseñar el Sistema

El sistema de ficheros distribuido basado en etiquetas adopta una **Arquitectura Orientada a Servicios (SOA)** jerárquica, estructurada en tres capas lógicas distintas: Interfaz/Gateway, Metadatos/Coordinación, y Almacenamiento/Datos. Esta estructura permite la escalabilidad independiente de cada componente, facilita la tolerancia a fallos por segregación de responsabilidades y optimiza el flujo de datos.

1.1. Organización del Sistema Distribuido

El sistema se organiza bajo un modelo de **tres capas lógicas** de servicio que interactúan mediante protocolos RPC (Remote Procedure Call). El diseño separa claramente la responsabilidad de la gestión de la lógica de negocio (Metadata Service) del almacenamiento físico de los datos (DataNode).

- **Capa de Acceso (Gateway):** Constituye el **punto de entrada unificado** para los clientes. Gestiona la autenticación, la validación sintáctica de los comandos y la orquestación de las operaciones que requieren la participación de múltiples DataNodes (ej., transferencias de archivos masivas). Actúa como un proxy inteligente.
- **Capa de Control (Metadata Service / NameNode):** El cerebro del sistema. Responsable de la **lógica de negocio** y la **coordinación**. Mantiene el índice global de etiquetas, gestiona el **Registro de Nodos Activos** y es la única entidad con autoridad para decidir la ubicación, replicación y eliminación de los datos.
- **Capa de Almacenamiento (DataNode / Storage Service):** Se encarga exclusivamente del **almacenamiento persistente** de los datos binarios. Sirve lecturas y escrituras de bajo nivel bajo la dirección del Metadata Service.

La comunicación entre servicios sigue un patrón de topología de red totalmente mallada (*fully meshed*) a nivel de servicio dentro de la red local, asegurando rutas de comunicación directas para optimizar el rendimiento.

1.2. Roles del Sistema

Se definen tres roles fundamentales que garantizan la disponibilidad y la integridad de los datos. La implementación de la funcionalidad de la interfaz (ej. **add file**) se distribuye: el **Gateway** maneja la coordinación del flujo, mientras que el **Metadata Service** contiene la lógica central de la consulta (*tag-query*) y la asignación de réplicas.

1.3. Distribución de Servicios en Docker Swarm

El sistema se desplegará en una **red local (LAN)** utilizando la infraestructura de contenedores **Docker Swarm**. Esto define un entorno de red de baja latencia y alta confiabilidad.

- **Red de Servicios ('overlay'):** Todos los servicios se desplegarán en la misma red overlay de Docker. Esto facilita el **Descubrimiento de Servicios** (mediante DNS interno) y permite la comunicación directa entre contenedores.
- **Registro de Nodos Activos:** Para la **localización precisa** de los datos, el sistema no dependerá del DNS de Swarm. En su lugar, el Metadata Service mantendrá un **Registro de Nodos Activos** propietario. Cada *DataNode* se registrará activamente con un **ID Único** y su **dirección IP/Puerto** específico.
- **Asignación de Datos:** Al subir un archivo (*add*), el Metadata Service utiliza su Registro para seleccionar **múltiples DataNodes al azar** (basado en la carga y el espacio disponible) y registra el mapeo preciso *Archivo ID* $\rightarrow$ *ID_Nodo(s)*, asegurando la redundancia.

Esta estrategia permite al Metadata Service monitorear el estado de cada instancia de *DataNode* de forma granular y activar los mecanismos de re-replicación en caso de fallo, cumpliendo con el requisito de no pérdida de datos.

2. Procesos o el Problema de Cuántos Programas o Servicios Posee el Sistema

La organización de los procesos y la elección del patrón de concurrencia son fundamentales para asegurar la alta disponibilidad y la eficiencia operativa del sistema, especialmente en el manejo intensivo de operaciones de Entrada/Salida (I/O).

2.1. Tipos de Procesos dentro del Sistema

Cada servicio (Gateway, Metadata Service, *DataNode*) se ejecuta como un proceso daemon (*demonio*) independiente dentro de su contenedor Docker. Aunque lógicamente se distinguen por su rol arquitectónico, su ejecución interna sigue un patrón uniforme:

- **Proceso Principal (Main Loop):** El único proceso del sistema operativo por contenedor. Su función es iniciar el **Bucle de Eventos** y monitorear la salud de los *workers* lógicos.
- **Workers Lógicos (Corrutinas/Tasks):** Unidades de concurrencia de muy bajo peso (no son hilos del sistema operativo). Se encargan de manejar una tarea específica no bloqueante:
 - **Worker de Red:** Maneja la recepción de peticiones RPC o la transferencia de datos.
 - **Worker de I/O de Disco:** Maneja la orquestación de lecturas/escrituras al almacenamiento local, delegando la espera al sistema operativo.
 - **Worker de Control:** Maneja tareas internas programadas, como el envío de *Heartbeats* o la ejecución de protocolos de *Gossip* para la sincronización de metadatos.

2.2. Organización o Agrupación de los Procesos

Se adopta una organización de **un Proceso por Contenedor**. Cada instancia de servicio (ej., `data-node-1`, `metadata-service-2`) corresponde a un único proceso principal del sistema operativo que maneja toda la concurrencia internamente.

- **Ventaja:** Se minimiza la sobrecarga (*overhead*) de la conmutación de contexto a nivel de sistema operativo y se aprovecha el aislamiento que proporciona Docker.
- **Agrupación:** Dado que los procesos son internamente no bloqueantes, no es necesario agrupar múltiples procesos principales en un solo contenedor. La escalabilidad se logra mediante la **replicación horizontal** del contenedor a través de Docker Swarm.

2.3. Tipo de Patrón de Diseño con Respecto al Desempeño

El patrón de diseño seleccionado es el **Modelo Asíncrono (*Async*) basado en Bucle de Eventos (*Event Loop*)**.

- **Justificación:** Los sistemas de ficheros son inherentemente **limitados por I/O (*I/O-Bound*)**. La ineficiencia principal surge al esperar que las operaciones lentas de red y disco finalicen.
- **Mecanismo:** En lugar de usar un modelo de **Hilos Bloqueantes (*Thread-per-request*)**, el bucle de eventos delega la espera de I/O al sistema operativo. Mientras el SO gestiona la operación de disco, el proceso principal continúa ejecutando otras tareas concurrentes.
- **Impacto:** Este enfoque permite a un único proceso mantener miles de conexiones concurrentes activas con un consumo de recursos considerablemente bajo, lo cual es crítico para la alta disponibilidad y el rendimiento en el entorno de red de baja latencia (LAN/Swarm). La concurrencia se logra a través de unidades de bajo peso (*coroutines* o *tasks*).

3. Comunicación o el Problema de Cómo Enviar Información Mediante la Red

La estrategia de comunicación se basa en la selección de protocolos específicos para optimizar cada tipo de tráfico (control, datos y monitoreo), aprovechando el entorno de baja latencia de la red local (LAN/Swarm).

3.1. Tipo de Comunicación

Se adopta un enfoque multicanal para la comunicación entre componentes:

- **gRPC (Protocolo Principal de Control):** Se utilizará para todos los comandos de control y metadatos (`list`, `delete`, actualización de índices, comunicación entre el `Gateway` y el `Metadata Service`).

- **Ventaja sobre REST:** gRPC utiliza **HTTP/2** para la multiplexación y **Protocol Buffers (Protobuf)** para una serialización binaria compacta. Esto resulta en una sobrecarga de comunicación significativamente menor y mayor velocidad de procesamiento que los protocolos basados en texto (JSON/REST), crítico para un sistema donde la eficiencia de la coordinación es primordial.
- **Streams gRPC (Transferencia de Datos):** Se empleará el **streaming bidireccional** de gRPC sobre HTTP/2 para el envío masivo de bloques de datos de los archivos.
 - **Alternativa (Sockets TCP Crudos):** Aunque los *sockets TCP crudos* ofrecen el control máximo sobre la transferencia de bits, su implementación es compleja. El uso de *Streams gRPC* ofrece seguridad (TLS), manejo de la conexión (HTTP/2) y control de flujo inherentes al protocolo, simplificando el desarrollo sin una penalización significativa en el entorno de LAN.
- **UDP Multicast (Monitoreo):** Utilizado exclusivamente para el protocolo de **Heartbeats (Latidos)**.
 - **Justificación:** UDP es sin conexión y de bajo *overhead*. **Multicast** permite que los **DataNodes** anuncien su estado a un grupo de escucha predefinido con una sola transmisión, minimizando la congestión de la red y facilitando la detección rápida de fallos y el descubrimiento de nuevos nodos.

3.2. Comunicación Cliente-Servidor y Servidor-Servidor

- **Cliente → Servidor (Gateway):** Comunicación externa mediante el protocolo definido para la interfaz pública (RPC o REST, dependiendo de la decisión final, aunque se prioriza gRPC). Esta comunicación debe ser autenticada y cifrada (TLS).
- **Servidor ↔ Servidor (Inter-Servicio):**
 - **Control y Metadata:** El Gateway, el **Metadata Service** y los **DataNodes** se comunican exclusivamente mediante **gRPC**.
 - **Datos:** Las transferencias de archivos (*file transfer*) entre el **Gateway** y los **DataNodes**, y la **replicación** entre **DataNodes** (patrón Push-Pull), utilizan **Streams gRPC** para eficiencia.

3.3. Comunicación entre Procesos y Patrones de Mensajes

La comunicación dentro del sistema está fuertemente definida por el patrón de replicación y distribución de tareas.

- **Patrón Push-Pull para Replicación:** Se utiliza una combinación de empuje y jale para la transferencia de datos.

- **Push (Empuje):** Utilizado inicialmente por el **Gateway** para enviar el archivo al conjunto de **DataNodes** primarios al momento de un `add file`.
- **Pull (Jale):** Utilizado por los **DataNodes** en situaciones de **re-replicación**. Un nodo que necesita restaurar una copia perdida (*Target*) solicita (*PULL*) activamente los bloques faltantes a un nodo sano (*Source*) que posee la réplica. Esto equilibra la carga al poner la responsabilidad de la transferencia en el nodo menos cargado.
- **Sincronización Asíncrona:** Dentro de cada proceso, la comunicación entre *workers* lógicos se gestiona mediante colas internas y el bucle de eventos, manteniendo el enfoque *non-blocking*.

4. Coordinación o el Problema de Poner Todos los Servicios de Acuerdo

La coordinación aborda los desafíos de la exclusión mutua para las escrituras críticas y la ordenación causal de eventos en un sistema distribuido propenso a particiones.

4.1. Acceso Exclusivo a Recursos y Condiciones de Carrera

Para evitar las condiciones de carrera donde múltiples clientes intentan modificar las etiquetas de un mismo archivo simultáneamente, se implementa un mecanismo de **Cerreo Distribuido** mediante la elección de un **Líder de Escritura de Metadatos**.

- **Estrategia:** Se garantiza que solo un nodo, el **Líder del Metadata Service**, sea responsable de serializar y autorizar todas las operaciones de modificación de etiquetas (`add-tags`, `delete-tags`, `delete file`).
- **Mecanismo:** Se utiliza el **Algoritmo de Bully (El Grandulón)** para la elección dinámica del Líder de Escritura. Este algoritmo asegura que, ante la caída del nodo coordinador, el nodo disponible con el ID más alto se auto-proclame rápidamente como el nuevo Líder, restableciendo la capacidad de serializar las escrituras sin depender de un punto único de fallo estático.

Este enfoque previene que las peticiones concurrentes lleguen a múltiples nodos y generen estados inconsistentes en el índice global de etiquetas.

4.2. Sincronización de Acciones (Ordenación de Eventos)

Para establecer un orden causal riguroso en las peticiones de modificación de metadatos, el sistema implementará **Relojes Lógicos** en conjunción con el Líder de Metadatos.

- **Relojes de Lamport:** Cada evento de modificación de etiquetas se marca con una marca de tiempo de Lamport. Esto permite al Líder de Metadatos resolver desempates: si dos peticiones concurrentes llegan, la petición con la marca de Lamport menor es la que se procesa primero, estableciendo un **orden total** de eventos.

4.3. Toma de Decisiones Distribuidas (Detección de Conflicto)

La toma de decisiones clave se centra en la capacidad de detectar y resolver conflictos derivados de la concurrencia y las particiones de red (*split-brain*).

- **Vectores de Versión (*Vector Clocks*):** En el nivel de datos (metadatos de cada archivo), se asocia un **Vector de Versión** con la colección de etiquetas.
- **Función:** Los Vectores de Versión no solo ordenan causalmente, sino que principalmente permiten la **detección de concurrencia**. Si dos actualizaciones de etiquetas provienen de diferentes particiones de red y ninguno de los vectores domina al otro, el sistema identifica un conflicto (o divergencia) que debe ser resuelto antes de la reconciliación de las réplicas.
- **Consenso en la Replicación:** El Metadata Service utiliza un enfoque de **Quorum** (el cual será detallado en la sección de Consistencia y Replicación) para decidir cuándo una operación de escritura se considera exitosa.

5. Nombrado y Localización o el Problema de Dónde se Encuentra un Recurso y Cómo Llegar al Mismo

El problema de la localización de recursos se resuelve mediante un **Directorio Replicado y Centralizado Lógicamente** mantenido por el Metadata Service.

5.1. Identificación de los Datos y Servicios

Todo recurso en el sistema debe tener un identificador único y persistente:

- **Archivos:** Se identifican mediante un **ID de Archivo Único** (UUID o un hash criptográfico del contenido). Este ID se utiliza para referenciar el archivo a través de todas las capas, desde la solicitud de metadatos hasta la recuperación de bloques de datos.
- **Nodos de Servicio:** Cada instancia de Metadata Service y DataNode se identifica mediante un **ID de Nodo Único** (ej., un número entero o UUID al arrancar).
- **Etiquetas:** Las etiquetas son identificadores de recursos al nivel de la interfaz, sirviendo como el principal medio de búsqueda y agrupación.

5.2. Ubicación de los Datos y Servicios

La información de ubicación se mantiene en la capa de control, utilizando una estructura de directorio centralizada replicada activamente.

- **Índice de Archivos (Mapeo de Ubicación):** El Metadata Service mantiene un mapeo estricto: *ID de Archivo* $\rightarrow$ *Lista de IDs de DataNode*. Esta lista de IDs representa las réplicas físicas del archivo.

- **Índice de Etiquetas (Mapeo de Metadatos):** Se mantiene un **Índice Invertido** replicado: *Etiqueta* → *Lista de IDs de Archivo*. Este índice permite resolver las *tag-queries* sin necesidad de acceder a la capa de almacenamiento.
- **Registro de Nodos Activos:** El **Metadata Service** mantiene un registro de la **dirección IP y Puerto** de cada **DataNode** activo, lo que permite la traducción de *ID de Nodo* a una dirección de red ejecutable para la transferencia directa de datos.

5.3. Localización de los Datos y Servicios

El proceso de localización se divide en dos fases que buscan optimizar el tráfico de red:

- **Localización de Metadatos (list, add-tags):** Se resuelve enteramente en la capa de **Metadata Service**. El **Metadata Leader** consulta el Índice de Etiquetas y el Índice de Archivos para obtener los metadatos y la lista de réplicas en una sola operación local.
- **Localización de Datos (Recuperación):**
 1. El **Gateway** solicita el ID del archivo al **Metadata Leader**.
 2. El **Metadata Leader** consulta el Índice de Archivos y, basándose en métricas de carga en tiempo real (proporcionadas por los *Heartbeats*), selecciona el **DataNode óptimo** para la lectura (ej. el menos ocupado).
 3. El **Metadata Leader** devuelve al **Gateway** la **dirección IP específica** del **DataNode** elegido.
 4. El **Gateway** establece una conexión **directa** con ese **DataNode** para la transferencia de datos masiva, liberando al **Metadata Service** de la tarea de transferencia de I/O.

6. Consistencia y Replicación o el Problema de Solucionar los Problemas que Surgen a Partir de Tener Varias Copias de un Mismo Dato en el Sistema

Para garantizar la durabilidad de los datos y la coherencia del índice de etiquetas, el sistema implementa una doble estrategia de consistencia: **Quorum** para los datos y **Gossip** para los metadatos.

6.1. Distribución de los Datos y Réplicas

- **Distribución:** Los archivos se distribuyen en múltiples **DataNodes** mediante un esquema de **asignación aleatoria con restricción por carga**, coordinado por el **Metadata Service**.

- **Replicación, Cantidad de Réplicas (N):** Se establece un factor de replicación base de $N = 3$. Cada archivo binario será replicado en tres **DataNodes** distintos para asegurar la durabilidad incluso ante el fallo de hasta dos nodos de almacenamiento.

6.2. Confiabilidad de las Réplicas de los Datos (Quorum)

La confiabilidad de los datos binarios se garantiza mediante la implementación de **Replicación Basada en Quorum** (N, R, W).

- **Parámetros de Quorum:** Se establecen los siguientes parámetros para garantizar la **Consistencia Fuerte** en las lecturas y escrituras:

$$N = 3, \quad W = 2, \quad R = 2$$

- **Mecanismo de Escritura (add file):** El **Gateway** solo considera la escritura como exitosa cuando recibe la confirmación de al menos $W = 2$ **DataNodes**. Si no se cumple el quorum de escritura, la operación se aborta para evitar inconsistencias y se devuelve un fallo.
- **Mecanismo de Lectura:** Para leer un archivo, el **Gateway** consulta $R = 2$ **DataNodes**. El sistema selecciona la versión del dato con el **Vector de Versión** más reciente.
- **Garantía de Consistencia:** La condición $R + W > N$ ($2 + 2 > 3$) se cumple, asegurando que el conjunto de nodos consultado en la lectura siempre se solape con el conjunto de nodos que participó en la última escritura, garantizando que el cliente siempre obtenga la versión más reciente del archivo.

6.3. Consistencia de los Metadatos y Manejo de Particiones

La coherencia del Índice Global de Etiquetas (metadatos) se maneja con protocolos de consistencia eventual de alto rendimiento.

- **Protocolo Gossip:** Las actualizaciones de etiquetas, una vez serializadas por el Líder de Metadatos, se propagan a través de la red utilizando un protocolo **Epidémico (Gossip)**.
 - **Ventaja:** A diferencia del *broadcast* total, el Gossip selecciona periódicamente un subconjunto **aleatorio** de vecinos (≤ 3) para intercambiar la información más reciente. Esto minimiza la carga de red inicial ($O(1)$ por nodo) y asegura una propagación exponencialmente rápida y tolerante a fallos.
- **Anti-Entropy y Árboles de Merkle:** Para la **reconciliación tras particiones**, el sistema utiliza **Árboles de Merkle** sobre el conjunto de metadatos. Cuando los nodos divergentes se reconectan, comparan los Hash Raíz de sus árboles. Si difieren, solo se intercambian las ramas (y bloques) de los datos que han cambiado, lo que permite una corrección de inconsistencias muy eficiente y la restauración de la uniformidad del índice.

7. Tolerancia a Fallas o el Problema de la Robustez del Sistema

La tolerancia a fallas es un requisito central del sistema, asegurando la disponibilidad continua y, lo más importante, la **no pérdida de datos** ante fallos de hardware o particiones de red.

7.1. Respuesta a Errores y Detección de Fallos

El sistema implementa un mecanismo de detección de fallos basado en latidos ligeros y *timeouts*.

- **Detección por Heartbeat:** Cada `DataNode` y `Metadata Service` envía periódicamente un pulso de vida (*Heartbeat*) a través de **UDP Multicast**. Esto permite que todos los pares detecten el estado de los demás con mínima sobrecarga.
- **Umbrales de Falla:**
 1. Si un nodo no recibe un *Heartbeat* de un par durante T_1 segundos (ej., 5 segundos), lo marca como **Sospechoso**.
 2. Si la falta de respuesta persiste por T_2 segundos (ej., 10 segundos), el `Metadata Service` lo declara **Muerto** y desencadena los mecanismos de re-replicación.
- **Replicación Activa de Servicios:** Los servicios críticos (`Metadata Service` y `Gateway`) operan con múltiples réplicas activas. En caso de caída de una instancia, Docker Swarm automáticamente reasigna el tráfico a las instancias disponibles.

7.2. Nivel de Tolerancia a Fallos Esperado

El diseño está optimizado para la **Disponibilidad** y la **Consistencia de Lectura** (vía Quorum), y ofrece una alta tolerancia a fallos de componentes individuales.

- **No Pérdida de Datos (Durabilidad):** Garantizada por el factor de replicación $N = 3$ y el uso de **Quorum** $W = 2$. El sistema puede tolerar la pérdida simultánea de hasta $N - W = 1$ réplica antes de que una escritura falle, y tolera la pérdida permanente de $N - 1 = 2$ `DataNodes` antes de la pérdida de un archivo.
- **Tolerancia a Partición (*Split-Brain*):** Durante una partición, el sistema prioriza la **Disponibilidad de Escritura**. Las particiones que mantengan un cuórum de `Metadata Services` continuarán aceptando modificaciones de etiquetas.

7.3. Fallos Parciales, Incorporación y Particiones

- **Auto-Healing (Recuperación Automática):** Cuando el `Metadata Service` detecta un `DataNode` como "Muerto", inmediatamente identifica todos los archivos cuyas réplicas han caído por debajo del factor $N = 3$. El Líder de Metadatos orquesta el proceso de **re-replicación automática**, instruyendo a un `DataNode` sano que copie los bloques de datos a un nuevo `DataNode` disponible.
- **Nodos Nuevos (*Scaling Out*):** Los nuevos `DataNodes` se incorporan simplemente registrándose con el `Metadata Service` (vía Heartbeat). Se ponen a disposición para recibir nuevas escrituras y ser **destinos de re-replicación** para archivos que necesiten restaurar su factor N .
- **Manejo de Particiones de Red:**
 1. **Escritura Divergente:** Se permite la modificación de metadatos en ambos lados de la partición (si se mantiene un cuórum local para la elección de un Líder).
 2. **Reconciliación:** Al reconectarse, los `Metadata Services` inician un proceso de **Anti-Entropy** asistido por **Árboles de Merkle**. Los conflictos son resueltos utilizando los **Vectores de Versión** asociados a los metadatos de cada archivo. La política de resolución por defecto será *Last Writer Wins* (LWW) basada en la causalidad determinada por el Vector de Versión.

8. Seguridad o el Problema de Qué Tan Vulnerable es el Diseño

La seguridad del sistema se aborda mediante la implementación de controles en las capas de comunicación, diseño arquitectónico y gestión de acceso, adhiriéndose al principio de defensa en profundidad.

8.1. Seguridad con Respecto al Diseño

El diseño de la arquitectura está guiado por el **Principio del Mínimo Privilegio (Principle of Least Privilege)**.

- **Segregación de Roles:** La separación estricta de responsabilidades en tres capas lógicas (`Gateway`, `Metadata Service`, `DataNode`) previene que el fallo o compromiso de un componente afecte a otros. Por ejemplo, un `DataNode` solo tiene permiso para leer y escribir datos binarios en respuesta a comandos RPC autorizados; no tiene la capacidad de modificar el índice de etiquetas global.
- **Control de Acceso Inter-Servicio:** Los servicios internos solo confían en las comunicaciones que provienen de los servicios inmediatamente anteriores en el flujo lógico (ej., un `DataNode` solo acepta comandos de

lectura/escritura del `Metadata Service` o del `Gateway` debidamente autenticados).

8.2. Seguridad con Respecto a la Comunicación

Se impone un cifrado obligatorio para proteger los datos y los metadatos en tránsito dentro de la red.

- **Cifrado TLS (Transport Layer Security):** Todas las conexiones basadas en `TCP/gRPC` (incluyendo la comunicación cliente-Gateway y la comunicación entre servicios internos) deben utilizar TLS. Esto mitiga el riesgo de interceptación (*eavesdropping*) y manipulación de datos en la red local.
- **Certificados:** Durante el desarrollo y despliegue en LAN, se utilizarán **certificados autofirmados** o una Autoridad de Certificación privada para establecer una raíz de confianza entre los contenedores Docker.
- **UDP Multicast:** Dado que los *Heartbeats* (UDP) no son críticos y son efímeros, se aceptará el riesgo de no cifrado para mantener el rendimiento y la ligereza del protocolo de detección de fallos.

8.3. Autorización y Autenticación

La gestión de acceso se basa en un sistema sin estado, garantizando la escalabilidad horizontal.

- **Autenticación (JWT):** Se utilizarán **JSON Web Tokens (JWT)** firmados criptográficamente para la autenticación de usuarios. Tras el *login* inicial, el usuario recibe un JWT que debe adjuntar a cada petición.
- **Autorización (Sin Estado):** El `Gateway` valida la firma del JWT utilizando una clave secreta compartida (o par de claves asimétricas). Esto permite que el sistema **valide la identidad y los permisos** del usuario en cada petición sin necesidad de consultar una base de datos central de usuarios, optimizando el rendimiento y la disponibilidad.
- **Autorización Inter-Servicio:** Las llamadas RPC entre el `Gateway` y el `Metadata Service` deben incluir credenciales de servicio (por ejemplo, tokens internos o certificados mutuos) que demuestren que el solicitante es una instancia de servicio legítima y no un proceso externo.